

LLD Practice Platform: Architecture & Research Notes

A Scalable, Domain-Driven System for Automated Low-Level Design Evaluation

Author: Technical Architecture Team | Status: Production Ready | Version: 1.0.0

1. Executive Summary & Problem Formulation

Low-Level Design (LLD) and Object-Oriented Design (OOD) form the foundational bridge between high-level distributed systems and clean, maintainable, production software. While algorithmic coding (e.g., LeetCode) benefits from deterministic unit test cases, evaluating software design has historically suffered from subjective, unstandardized human feedback. Engineers often struggle to articulate class responsibilities, encapsulate contracts, anticipate extensibility requirements, or identify failure edge cases.

The **LLD Practice Platform** resolves this challenge by introducing an automated, structured practice ecosystem. Users formulate solutions through a 7-stage architectural decomposition process. Submissions are processed through a deterministic state machine and evaluated against an 8-dimensional rubric using dual evaluation engines: a high-throughput *Rule-Based Compliance Engine* and a nuanced *AI Architectural Reviewer*.

Architectural Core Principle: Separate the mutable evaluation algorithms (strategies) from the immutable domain lifecycle (state machine). This enables pluggable evaluation backends—ranging from deterministic rule engines to multimodal LLMs—without affecting attempt tracking, persistence, or client communication contracts.

2. Domain-Driven Design & Core Entities

The platform is organized strictly around DDD (Domain-Driven Design) tenets. Core business rules reside in pure, zero-dependency domain models, preventing framework lock-in and guaranteeing total testability:

Domain Entity	Responsibility & Encapsulation	Key Invariants
Problem	Encapsulates problem identity, difficulty tier, title, description, and atomic requirements.	Immutable once loaded into catalog.
Attempt	Tracks user progress through a guarded Finite State Machine (FSM). Owns timestamps and IDs.	State transitions are strictly one-directional.
TextSubmission	Specializes abstract Submission base class to hold 7 structured design fields.	Submission base class is abstract and uninstantiable.
Evaluation	Aggregates 8 Feedback items, computes overall normalized score (0-100), tracks completion.	Overall score = sum of 8 criteria scores * 1.25.
Feedback	Atomic assessment unit containing score (1-10), evidence, concern, suggestion, and confidence.	All 5 diagnostic dimensions must be non-null.

3. Deterministic Attempt Lifecycle & State Machine

A critical vulnerability in practice platforms is inconsistent state progression (e.g. evaluating before submission, or mutating an attempt after final scoring). We solved this using a formal Finite State Machine (FSM):

```
[ DRAFT ] --( submitAttempt )--> [ SUBMITTED ] | ( evaluateAttempt ) v [ COMPLETED ] <--(
completeAttempt )-- [ EVALUATING ]
```

State Machine Invariants:

- DRAFT: Represents an in-flight session. Any attempt to directly evaluate or complete throws a domain violation error.
- SUBMITTED: Design payload is frozen; transition to EVALUATING is triggered immediately by the submission pipeline.
- EVALUATING: Evaluator strategy runs in-process or via external worker.
- COMPLETED: Terminal state. Re-submitting an already completed attempt is strictly rejected with an HTTP 400 error.

4. Dual Evaluation Strategy & 8-Criterion Rubric

The platform implements the GoF **Strategy Pattern**. An abstract Evaluator class defines the uniform evaluation contract. The EvaluationService acts as the execution context, maintaining an extensible registry of evaluators:

Rubric Criterion	Target Dimension	Diagnostic Focus
1. Requirement Understanding	Functional scope & boundaries	Checks if core features and user stories are clearly identified.
2. Class Responsibilities	Single Responsibility Principle (SRP)	Evaluates whether classes avoid 'God Object' antipatterns.
3. Coupling & Cohesion	Inter-class dependency management	Assesses composition vs inheritance and relationship cardinality.
4. Encapsulation & Interfaces	Data hiding & contract definition	Ensures state is protected through interfaces and accessors.
5. Abstraction & Patterns	Design patterns (Strategy, Factory, etc.)	Validates deliberate architectural pattern choices.
6. Extensibility	Open/Closed Principle (OCP)	Analyzes how easily new requirements can be added.
7. Edge Cases & Testability	Failure modes & concurrent safety	Reviews race conditions, capacity limits, and mockability.
8. Explanation Quality	Architectural rationale & trade-offs	Assesses reasoning clarity, assumptions, and justification.

5. Empirical Verification & Test Suite Performance

A rigorous, multi-tier automated test suite validates the entire stack. All unit, domain, and API end-to-end tests execute synchronously via `server/testRunner.js` with 100% pass rate:

Test Layer	Scope & Scenarios	Result
Domain Unit Tests	Attempt, Evaluation, Feedback, Problem, Rubric, Submission invariants	10 / 10 Passed
Evaluator Unit Tests	RuleBasedEvaluator & AIEvaluator scoring and diagnostic accuracy	2 / 2 Passed
Service Unit Tests	EvaluationService dispatch, PracticeService attempt lifecycle	2 / 2 Passed
E2E Integration Flows	Parking Lot, Vending Machine, Elevator System, Library Management	4 / 4 Passed
Error Injection Tests	Invalid problem (400), invalid attempt (400/404), route 404, re-submit (400)	5 / 5 Passed

6. Future Research & Scalability Roadmap

- 1. Dynamic Code Execution Sandbox:** Integrate WebAssembly (WASM) or Docker-isolated Node/Java runtimes to compile and test user code against live mock harness suites.
- 2. UML & Diagrammatic Parsing:** Extend the Submission abstract hierarchy to support Mermaid, PlantUML, and drag-and-drop class diagrams with semantic AST relationship validation.
- 3. Vectorized Design Benchmarking:** Embed past submissions into a vector database (e.g. pgvector / Qdrant) to calculate semantic similarity scores against canonical industry solutions.

Conclusion: The LLD Practice Platform establishes an architectural blueprint for automated, high-fidelity system design training. By enforcing strict domain invariants, decoupling evaluation engines via Strategy Pattern, and delivering instant 8-dimensional rubric feedback, the platform empowers engineers to transition from ad-hoc coding to principled, production-grade object-oriented design.